

SEIF Developer Handover & Technical Learning Guide

A structured guide for taking technical ownership of the existing SEIF project

Architecture • Authentication • Database • Business Flows • Deployment • Debugging

1. Purpose

Use this guide to understand the existing SEIF codebase from a developer's perspective. The goal is not to memorize every file, but to understand the major flows well enough to troubleshoot, modify and deploy the system safely.

Learning principle: trace complete flows from frontend → API → backend → database/external service → response.

2. High-Level Architecture

React Frontend → REST API → Node/Express Backend → MySQL/MariaDB

Supporting systems include JWT authentication, AWS S3, SMTP/Nodemailer, Redis, Socket.IO/WebSockets, cron jobs, Sentry, file processing, PDF generation and reporting.

Typical request flow

- React page/action triggers an API call.
- Axios/API client sends the request.
- Express route receives it.
- Authentication middleware validates the token.
- Role/authorization middleware checks access.
- Controller receives the request.
- Service performs business logic.
- Model/database layer reads or updates MySQL.
- External services may be called.
- Response returns to the frontend.

3. Authentication — Highest Priority

This is the first area to learn because login, password and role-access issues are common production problems.

Backend: auth.service.js, auth.controller.js, auth.routes.js, auth.middleware.js, role.middleware.js, User.model.js, user.service.js

Frontend: LoginPage.jsx, auth API/service, authSlice.js, useAuth.js, ProtectedRoute.jsx, ForcedPasswordChangeGuard.jsx

4. Partner Authentication Lifecycle

- Admin creates a Partner through Organization Management.
- Partner record is created.
- PARTNER user is created and linked through partner_id.
- Temporary password is generated.
- Intended process sends credentials to the organization's email.
- Partner logs in.
- Backend validates email/password using the users table and bcrypt.

- JWT access/refresh tokens are generated.
- Frontend establishes authentication state.
- Role-based access controls the Partner's APIs/pages.
- If `must_change_password` is true, forced password change is required.

Important handover point: the Partner creation flow appears to generate credentials while the welcome-email call was noted as disabled/commented. Confirm whether this is intentional and whether production email delivery is active.

5. Password Reset & Credentials

- Admin reset: generate temporary password → hash → update user → enable `must_change_password` → user logs in → forced password change.
- Partner credential resend: locate Partner user → generate new temporary password → update password → enable forced change → attempt credential email.
- Understand exactly what happens when email delivery fails.

6. Roles & Authorization

Roles used in the project include `SUPER_ADMIN`, `ADMIN`, `PARTNER`, `SEIF_READONLY`, `SEIF_READONLY_DOWNLOAD` and `ESSCI`.

Understand both frontend route protection and backend authorization. Backend authorization is the security boundary.

7. Database Architecture

Start with the core entities instead of memorizing the full schema.

Table/entity	Purpose
users	Authentication, role, status, Partner relationship
partners	Partner organization/business record
centers	Partner-associated centers
courses	Course/configuration information
batches	Training/batch information
students	Student records
data_uploads	Upload tracking/history
uploaded_*	Processed/uploaded data
notifications	Application notifications
audit_logs	Activity/history tracking

Critical relationship: `users.partner_id` → `partners.id`. Understand how this controls Partner data access.

8. Data Upload Flow

- Partner uploads CSV/Excel data.
- Backend validates the upload.
- Invalid records/files are reported.
- Valid data is processed and stored.
- Upload history/status is recorded.
- Admin reviews the submission.
- Admin approves or rejects.
- Partner corrects rejected data where applicable.
- Partner resubmits.
- Approved data becomes available to reports/analytics.

Debugging focus: identify the upload API, controller/service, validation logic, database tables, status transitions and frontend page.

9. Certification / ESSCI

Learn the complete lifecycle: upload → validation → review → ESSCI processing → certificate/PDF → approval → archive/reporting. Confirm where original, processed, final and archived files are stored.

10. Refurbishment / Upgradation

Learn one real request end-to-end: Partner request → eligibility → package/selection → admin review → approval/rejection → completion → attachments/photos → notifications → reporting.

11. Notifications & Email

Understand database notifications, email notifications, scheduled notifications and Socket.IO notifications separately. For a missing notification, trace creation → queue/sending → delivery → frontend display.

12. File Storage

Map every major file type to its storage location: CSV/Excel uploads, certification PDFs, refurbishment documents/images, reports and attachments. Confirm local storage vs S3 and access rules.

13. Scheduled Jobs & WebSockets

Ask which cron jobs run in production, their schedules, what they change/send, and what happens after a restart. Also identify which UI features use Socket.IO.

14. Environment & Production Configuration

- Database host/port/user/password/name
- JWT secrets and token expiry
- Frontend/backend URLs
- AWS/S3 configuration
- Redis configuration

- SMTP/email configuration
- Sentry/monitoring configuration
- Deployment-specific variables

Security check: confirm production does not rely on development fallback secrets or default values.

15. Database Migrations

The repository contains many migration/maintenance scripts. Confirm which have already run in each environment and the official development → staging → production migration process. Do not execute historical scripts blindly.

16. Deployment & Production Ownership

- Where are frontend, backend and MySQL hosted?
- Where are S3 files and Redis hosted?
- Where is SMTP configured?
- How are environment variables managed?
- How is the backend restarted?
- Where are application/server logs?
- How is deployment performed?
- What is the rollback process?
- How are database backups created/restored?

17. Old Code & Technical Debt

Identify backup/old/optimized files and historical migrations. Ask which files are active production code and which are obsolete. Also identify known fragile modules.

18. Standard Bug-Debugging Process

- Reproduce the issue and record exact steps.
- Check browser console and Network tab.
- Identify the API endpoint and HTTP status.
- Inspect request/response payloads.
- Trace route → middleware → controller → service.
- Check database queries/data.
- Check S3, SMTP, Redis or Socket.IO when relevant.
- Check server logs and monitoring.
- Fix the root cause and test affected roles.
- Document the fix and any migration/deployment requirement.

19. Questions for the Consultant

Authentication

1. What is the complete authentication flow from login to dashboard?
2. What is the difference between access and refresh tokens?
3. Where are tokens stored and refreshed?
4. How does logout work?
5. How are inactive/suspended users handled?
6. How does forced password change work?
7. How are Partner credentials created, reset and resent?
8. Where does the Partner temporary password go?
9. Is the Partner welcome email working in production?
10. What can each role access and where is each permission enforced?
11. What are the most important database tables and relationships?
12. Can you demonstrate one complete Upload → Review → Approve/Reject → Resubmit flow?
13. Can you demonstrate the complete Certification/ESSCI lifecycle?
14. What exactly should I check if production goes down?
15. What are the top 10 issues you have previously handled?
16. What are the most fragile parts of the code?
17. Which files/scripts should I avoid changing or executing without checking first?

20. Recommended 5-Day Learning Plan

Day 1: Architecture + Authentication + Partner login/password lifecycle

Day 2: Database + Organization Management + Partner/Centre/Course relationships

Day 3: Data Upload + Validation + Review + Approval/Resubmission

Day 4: Certification/ESSCI + Refurbishment/Upgradation + Notifications

Day 5: Deployment + Production + Logs + Backup/Recovery + Real bug debugging

21. Personal SEIF Knowledge Base Template

- Purpose
- Frontend pages/components
- API endpoints
- Routes
- Controllers
- Services
- Database tables
- External services
- Roles/permissions
- Important files
- Common bugs
- How to debug

- Deployment/migration requirements

22. Immediate Priority — Partner Login

Make this the first complete flow you learn with the consultant because it is also an active project issue.

- Create Partner in Organization Management.
- Confirm Partner and user records in the database.
- Confirm partner_id relationship.
- Confirm temporary password generation.
- Confirm credential email delivery.
- Attempt Partner login.
- Trace the login API.
- Verify password validation and JWT creation.
- Verify role/permission checks.
- Verify forced password change if applicable.
- Verify Partner dashboard/API access.

Target outcome: you should be able to diagnose a Partner login failure from the browser through the backend and database without depending on the previous developer.